

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL Aplikasi Web dan Mobile Manual Praktikum – React
Routing & Multi-Page App + Integrasi API**

Nama Mahasiswa : Wianda Sukandari
NIM : 23051430002
Kelas : A
Tanggal Praktikum : 02 Mei 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul Aplikasi Web dan Mobile Manual Praktikum (Pertemuan 11)
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	02 Mei 2026
Nama Mahasiswa	Wianda Sukandari
NIM	23051430002
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react router-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging

3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React Router DOM	Versi 6+	Routing & Navigasi Halaman
5	JSONPlaceholder API	API Online	Simulasi Fetch Data API

D. DASAR TEORI

D.1 Konsep Utama

Pada modul ini saya belajar tentang routing pada React menggunakan react-router-dom. Routing digunakan supaya aplikasi dapat memiliki beberapa halaman seperti dashboard, inventory, dan laporan tanpa harus membuat aplikasi terpisah. Dengan adanya routing, perpindahan halaman menjadi lebih cepat dan tidak perlu reload browser.

Saya juga belajar menggunakan beberapa komponen penting seperti BrowserRouter, Routes, Route, dan Link. BrowserRouter digunakan untuk mengaktifkan sistem routing, Route digunakan untuk menentukan halaman berdasarkan URL, sedangkan Link dipakai untuk berpindah halaman dengan lebih mudah.

Selain itu, modul ini membantu saya memahami bagaimana aplikasi web modern dibuat secara modular. Setiap halaman dipisahkan ke file masing-masing supaya kode lebih rapi, mudah dibaca, dan lebih mudah dikembangkan ketika aplikasi semakin besar.

D.2 Keterkaitan dengan Teknik Industri

Materi pada modul ini berhubungan dengan Teknik Industri karena di dunia industri banyak sistem yang menggunakan aplikasi web untuk mengelola data dan memantau proses kerja. Dengan adanya routing, aplikasi bisa dibagi menjadi beberapa halaman seperti dashboard, inventori, laporan, dan monitoring produksi sehingga tampilan lebih rapi dan mudah digunakan.

Contoh penerapannya yaitu pada sistem inventori di pabrik. Operator gudang dapat membuka halaman inventory untuk melihat stok bahan baku, lalu membuka halaman detail barang untuk melihat informasi item tertentu. Selain itu, data juga bisa diambil langsung dari database atau API sehingga informasi yang ditampilkan lebih cepat dan mudah diperbarui.

Materi ini juga membantu dalam pembuatan sistem monitoring produksi atau laporan kualitas di perusahaan. Dengan aplikasi berbasis web, proses pengolahan data menjadi lebih efisien, mengurangi pencatatan manual, dan mempermudah pengambilan keputusan dalam kegiatan industri.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

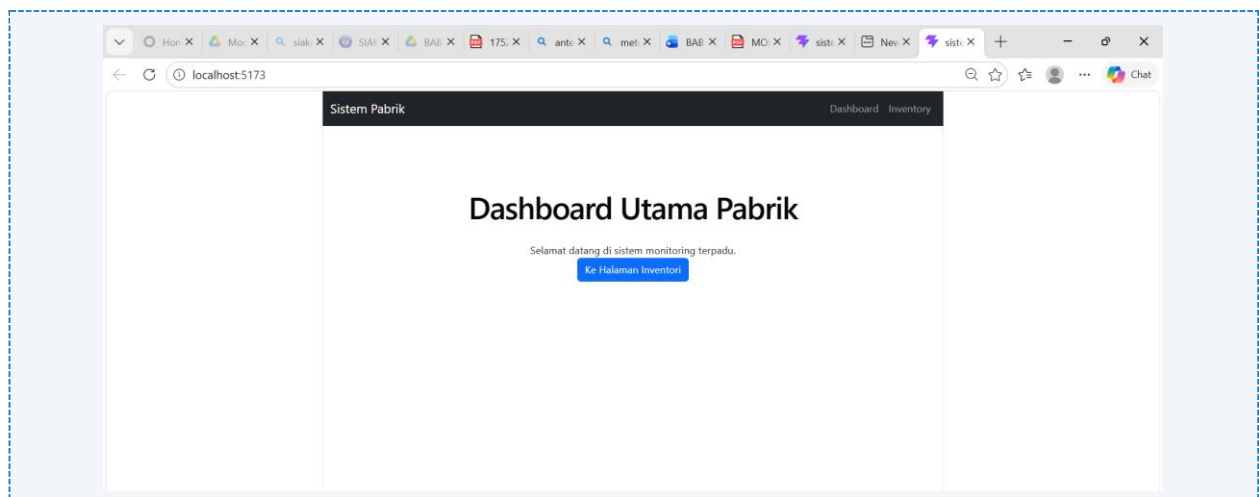
E.1 Langkah 1 – Setup Router

Pada langkah ini saya menambahkan router pada aplikasi React menggunakan react-router-dom. Router digunakan supaya aplikasi bisa berpindah halaman seperti Dashboard, Inventory, dan halaman Error tanpa perlu reload browser. Saya juga membungkus aplikasi dengan BrowserRouter di file main.jsx dan menambahkan beberapa Route di App.jsx agar setiap URL bisa menampilkan halaman yang sesuai. Dengan cara ini, navigasi aplikasi jadi lebih cepat dan nyaman digunakan.

```
/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)
```

Screenshot Hasil:



Gambar E.1 – Setup Router

E.2 Langkah 2 – Membuat Halaman (Pages)

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
Isi `src/Halaman/Dashboard.jsx`:
import React from 'react';
import { Link } from 'react-router-dom';
function Dashboard() {
  return (
    <div className="p-5">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
      <Link to="/inventori" className="btn btn-primary">
        Ke Halaman Inventori
      </Link>
    </div>
  );
}
export default Dashboard;
```

```
Isi `src/Halaman/Inventori.jsx`:
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
function Inventori() {
  const [products, setProducts] = useState([]);
  // Fetch Data saat komponen mount
  useEffect(() => {
    fetch('https://jsonplaceholder.typicode.com/posts')
      .then(res => res.json())
      .then(data => {
        // Ambil hanya 5 data pertama untuk contoh
        setProducts(data.slice(0, 5));
      })
      .catch(err => console.log(err));
  }, []);
  return (
    <div className="container mt-4">
      <h1>Data Inventori Bahan Baku</h1>
      <Link to="/" className="btn btn-secondary mb-3">Kembali ke
Dashboard</Link>
      <table className="table table-striped">
        <thead>
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>
        <tbody>
          {products.map((item) => (
```

```

        <tr key={item.id}>
          <td>{item.id}</td>
          <td>{item.title}</td> { /* Menggunakan title
sebagai simulasi nama bahan */}
          <td><span className="badge bg-
success">Available</span></td>
        </tr>
      </tbody>
    </table>
  </div>
);
}
export default Inventori;

```

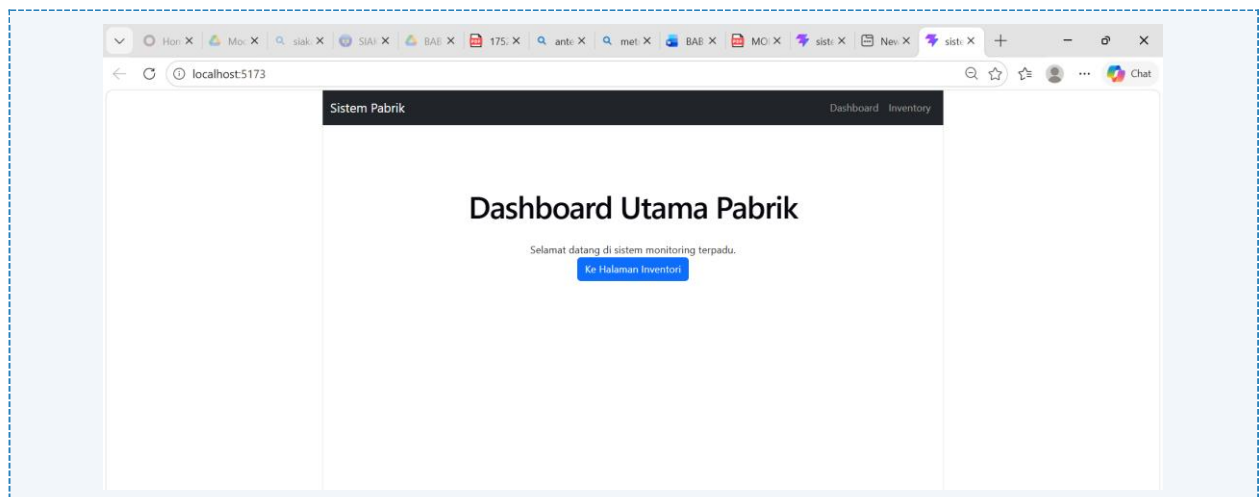
Isi `src/Halaman/NotFound.jsx`:

```

import React from 'react';
import { Link } from 'react-router-dom';
function NotFound() {
  return (
    <div className="text-center mt-5">
      <h1 className="display-1">404</h1>
      <p>Halaman tidak ditemukan dalam sistem pabrik.</p>
      <Link to="/" className="btn btn-dark">Kembali ke Home</Link>
    </div>
  );
}
export default NotFound;

```

Screenshot Hasil:

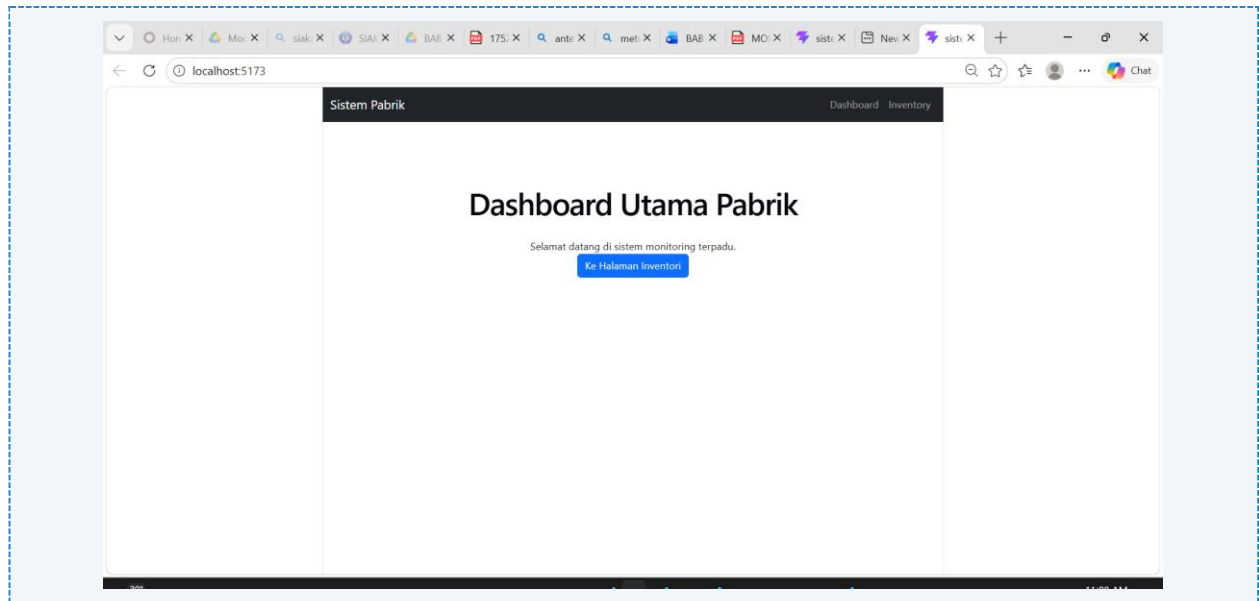


Gambar E.2 – Membuat Halaman (Pages)

E.3 Langkah 3 – Mengatur Route di App.jsx

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventory from './Halaman/Inventory';
import NotFound from './Halaman/NotFound';
// import Navbar from './Komponen/Navbar'; // Anda bisa buat navbar
// sendiri
// Buat Navbar.jsx sederhana untuk navigasi
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
      <div className="container">
        <a className="navbar-brand" href="/">Sistem Pabrik</a>
        <div className="navbar-nav">
          <a className="nav-link" href="/">Dashboard</a>
          <a className="nav-link" href="/inventory">Inventory</a>
        </div>
      </div>
    </nav>
  )
}
function App() {
  return (
    <div>
      <Navbar />
      <Routes>
        {/* Route yang tepat akan di-render */}
        <Route path="/" element={<Dashboard />} />
        <Route path="/inventory" element={<Inventory />} />
        {/* Route untuk semua path lainnya (404) */}
        <Route path="" element={<NotFound />} />
      </Routes>
    </div>
  );
}
export default App;
```

Screenshot Hasil:



Gambar E.3 – Mengatur Route di App.jsx

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dynamic Route

Pertanyaan / Instruksi Latihan:

Tambahkan Route untuk detail item. Misal path `/inventori/:id`. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

Copy kode diatas (Latihan 1 sampai 3) kemudian tambahkan file baru `DetailItem.jsx` dan diisi dengan kode berikut

```
import React from 'react';
import { useParams, Link } from 'react-router-dom';

function DetailItem() {
  const { id } = useParams();

  return (
    <div className="container mt-4">
      <h1>Detail Item Inventory</h1>

      <div className="card p-4 shadow-sm">
        <h3>ID Item: {id}</h3>

        <p>
          Ini adalah halaman detail item inventory dengan ID
          {id}.
        </p>
      </div>
    </div>
  );
}
```



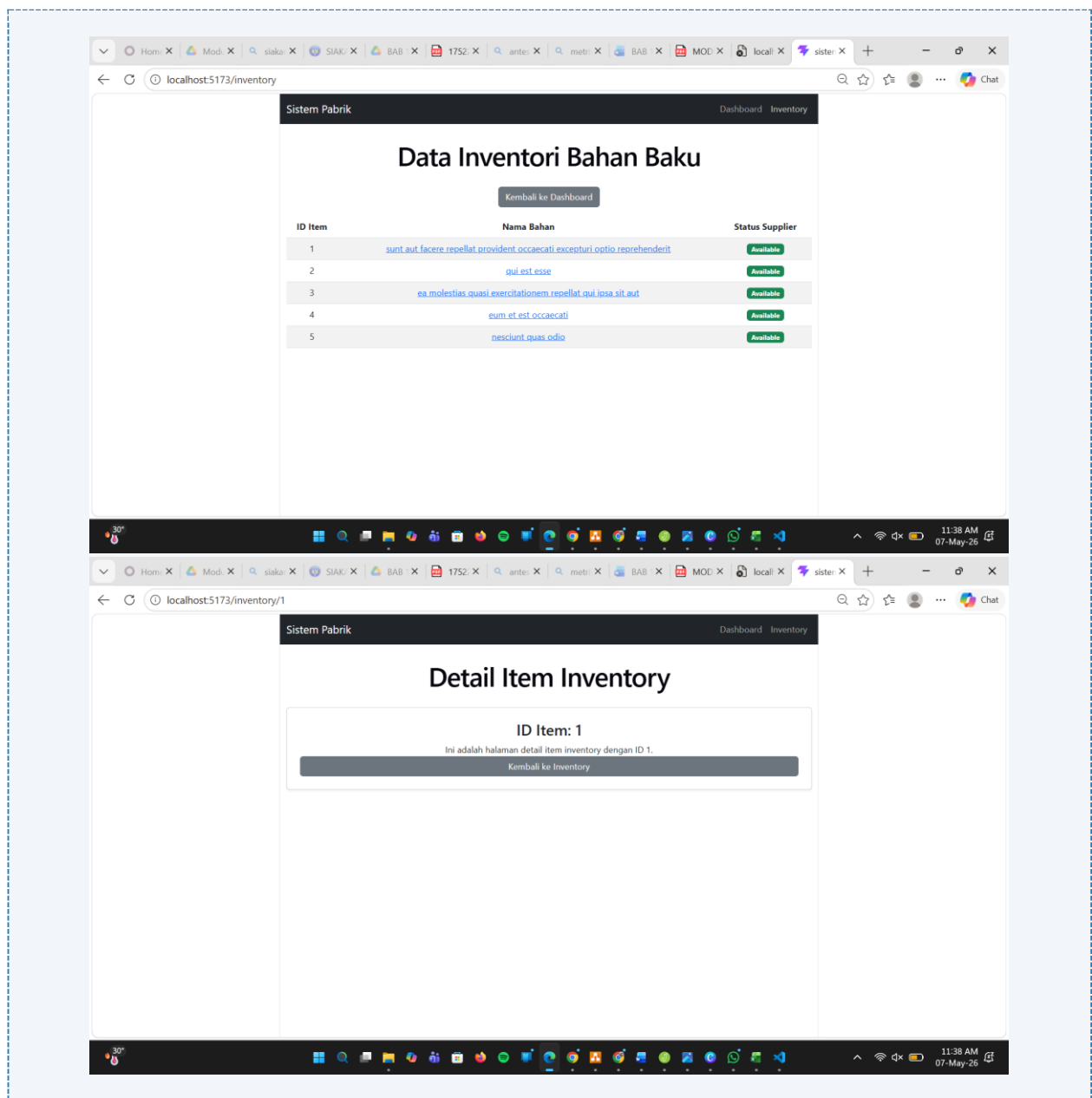
```

    </p>

    <Link to="/inventory" className="btn btn-secondary">
      Kembali ke Inventory
    </Link>
  </div>
</div>
);
}

export default DetailItem;

```



Gambar F.1 – Dynamic Route

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventory() {

  // State data produk
  const [products, setProducts] = useState([]);

  // State loading
  const [loading, setLoading] = useState(true);

  // Fetch data
  useEffect(() => {

    // Aktifkan loading
    setLoading(true);

    // Delay 2 detik supaya loading terlihat
    setTimeout(() => {

      fetch('https://jsonplaceholder.typicode.com/posts')

        .then((res) => res.json())

        .then((data) => {

          // Ambil 5 data pertama
          setProducts(data.slice(0, 5));

          // Loading selesai
          setLoading(false);
        })

        .catch((err) => {

          console.log(err);

          // Jika error loading dimatikan
          setLoading(false);
        });
    });
  });
}
```

```

    }, 2000);

    }, []);

    return (
      <div className="container mt-4">

        <h1>Data Inventory Bahan Baku</h1>

        <Link to="/" className="btn btn-secondary mb-3">
          Kembali ke Dashboard
        </Link>

        {/* Conditional Rendering Loading */}
        {loading ? (

          <div className="alert alert-info">
            <h4>Memuat data...</h4>
          </div>

          ) : (

            <table className="table table-striped table-bordered">

              <thead className="table-dark">
                <tr>
                  <th>ID Item</th>
                  <th>Nama Bahan</th>
                  <th>Status Supplier</th>
                </tr>
              </thead>

              <tbody>

                {products.map((item) => (

                  <tr key={item.id}>

                    <td>{item.id}</td>

                    <td>
                      <Link to={`/${inventory}/${item.id}`}>
                        {item.title}
                      </Link>
                    </td>

                    <td>
                      <span className="badge bg-success">
                        Available
                      </span>

```

```
        </td>

      </tr>

    ) ) }

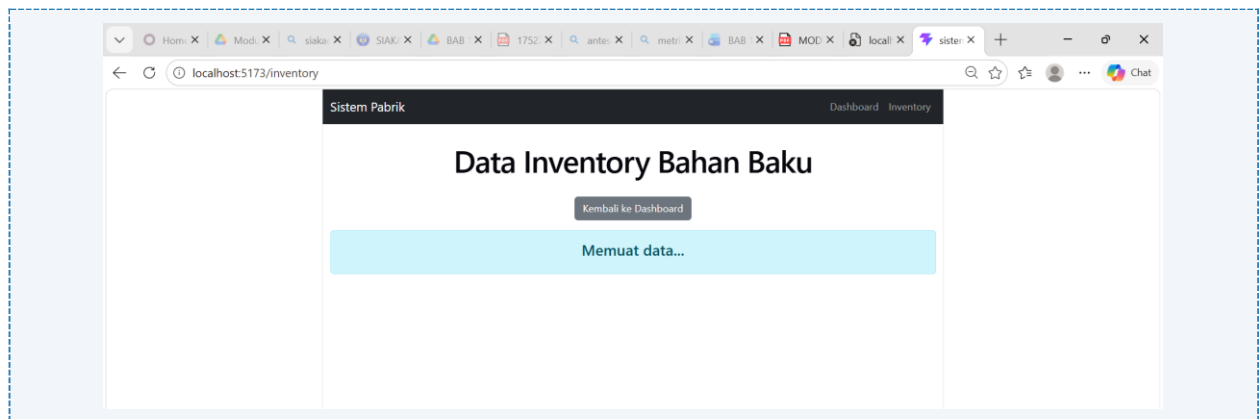
  </tbody>

</table>

) }

</div>
);
}

export default Inventory;
```



Gambar F.2 – Loading State

F.3 Tugas Proyek Mini – Navigasi Modular

Deskripsi proyek mini:

- Buat halaman baru bernama 'LaporanKualitas.jsx'. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser.

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini, saya membuat halaman baru bernama LaporanKualitas.jsx untuk menampilkan data cacat produk menggunakan array manual (mock data). Setelah itu, saya menambahkan route baru pada App.jsx agar halaman bisa diakses melalui URL /laporan-kualitas. Saya juga menambahkan menu navigasi pada Navbar menggunakan Link dari react-router-dom.

Agar perpindahan halaman lebih cepat dan tidak reload browser, saya menggunakan konsep SPA (Single Page Application) dengan React Router. Dengan cara ini, pengguna bisa berpindah dari Dashboard, Inventory, dan Laporan Kualitas dengan lebih mulus. Selain itu, saya menggunakan komponen tabel Bootstrap supaya tampilan data lebih rapi dan mudah dibaca.

Kode Utama:

```
import React from 'react';
import { Link } from 'react-router-dom';

function LaporanKualitas() {

  // Mock data cacat
  const dataCacat = [
    {
      id: 1,
      nama: "Produk Retak",
      jumlah: 12
    },
    {
      id: 2,
      nama: "Warna Tidak Sesuai",
      jumlah: 7
    },
    {
      id: 3,
      nama: "Ukuran Tidak Presisi",
      jumlah: 5
    }
  ];

  return (
    <div className="container mt-4">

      <h1>Laporan Kualitas Produk</h1>

      <Link to="/" className="btn btn-secondary mb-3">
        Kembali ke Dashboard
      </Link>

      <table className="table table-bordered table-striped">
```

```

<thead className="table-dark">
  <tr>
    <th>ID</th>
    <th>Jenis Cacat</th>
    <th>Jumlah</th>
  </tr>
</thead>

<tbody>

  {dataCacat.map((item) => (

    <tr key={item.id}>
      <td>{item.id}</td>
      <td>{item.nama}</td>
      <td>{item.jumlah}</td>
    </tr>

  ))}

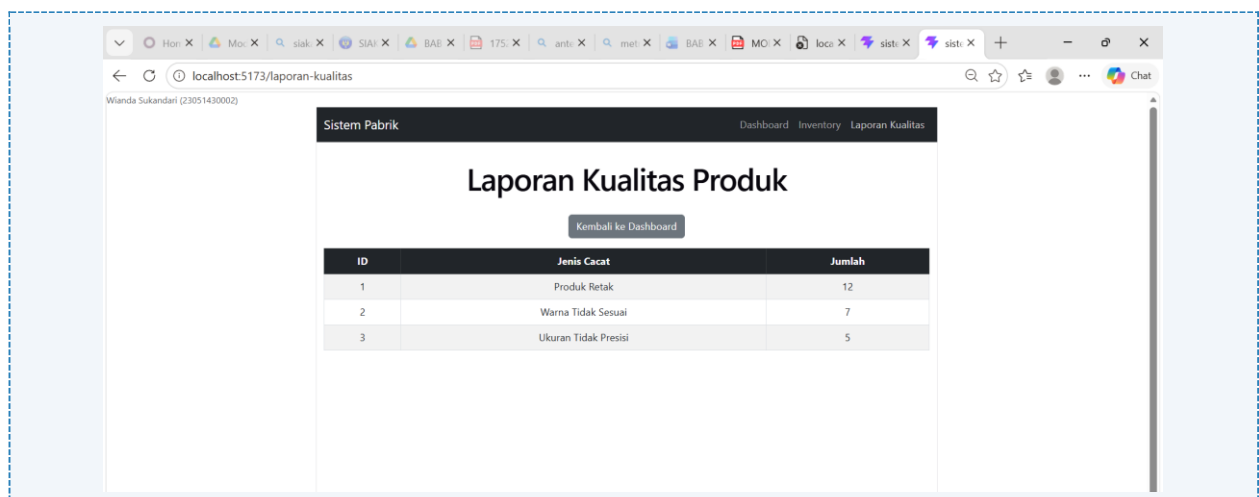
</tbody>

</table>

</div>
);
}

export default LaporanKualitas;

```



Gambar F.3 – Navigasi Modular

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum yang diperoleh sudah sesuai dengan teori yang dipelajari pada modul React Router dan Fetch API. Pada bagian routing, aplikasi berhasil berpindah halaman menggunakan react-router-dom tanpa melakukan reload browser. Hal ini menunjukkan bahwa konsep Single Page Application (SPA) berjalan dengan baik. Route seperti /inventory, /inventory/:id, dan /laporan-kualitas dapat menampilkan komponen yang berbeda sesuai URL yang dipilih.

Selain itu, penggunaan Link juga sudah sesuai teori karena navigasi dilakukan secara client-side sehingga perpindahan halaman terasa lebih cepat dan ringan dibanding menggunakan tag <a>. Dynamic route pada halaman detail item juga berhasil menampilkan data berdasarkan parameter id dari URL menggunakan useParams().

Pada bagian Fetch API dan loading state, data dari API berhasil diambil menggunakan fetch() di dalam useEffect(). Saat data masih diproses, teks “Memuat data...” muncul karena state loading bernilai true, lalu berubah menjadi tabel data setelah fetch selesai. Hal ini membuktikan bahwa penggunaan state dan effect pada React sudah berjalan sesuai konsep yang dipelajari.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Halaman tidak berpindah dengan benar saat klik menu Navbar	Mengganti tag <a> menjadi <Link> dari react-router-dom
Loading state tidak muncul	Menambahkan setLoading(true) sebelum fetch dan setLoading(false) setelah data berhasil dimuat
Route detail item /inventory/:id tidak berjalan	Menambahkan route baru pada App.jsx dan menggunakan useParams() untuk membaca parameter URL

G.3 Kaitan dengan Penerapan di Industri

Materi pada modul ini sangat relevan dengan dunia Teknik Industri, terutama dalam pengembangan sistem monitoring dan manajemen data produksi berbasis web. Konsep routing dapat digunakan untuk membuat aplikasi dengan banyak halaman seperti dashboard produksi, inventori bahan baku, laporan kualitas, dan data supplier tanpa perlu reload halaman sehingga aplikasi menjadi lebih cepat dan efisien.

Penggunaan Fetch API juga penting dalam industri karena data pada sistem manufaktur biasanya diambil dari database atau server secara real-time. Contohnya pada sistem monitoring gudang, data stok bahan baku dapat otomatis diperbarui dari server dan langsung ditampilkan pada halaman inventori. Selain itu, loading state membantu memberikan informasi kepada pengguna bahwa sistem sedang memproses data sehingga aplikasi terasa lebih responsif dan nyaman digunakan.

Dynamic route juga bermanfaat dalam sistem industri untuk menampilkan detail data tertentu. Contohnya, ketika operator memilih salah satu produk pada halaman inventori, sistem dapat langsung membuka halaman detail produk berdasarkan ID item tersebut. Dengan demikian, proses pencarian informasi menjadi lebih cepat dan terstruktur.

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan konsep React Router untuk membuat navigasi antar halaman seperti Dashboard, Inventory, dan Laporan Kualitas tanpa reload browser menggunakan metode SPA (Single Page Application).
2	Penggunaan dynamic route pada path <code>/inventory/:id</code> berhasil digunakan untuk menampilkan halaman detail item berdasarkan ID yang dipilih dari tabel inventory.
3	Fetch API dan hook <code>useEffect()</code> berhasil digunakan untuk mengambil data dari API eksternal, kemudian ditampilkan ke dalam tabel inventory secara otomatis.
4	State loading berhasil diterapkan untuk menampilkan indikator “Memuat data...” saat proses pengambilan data berlangsung sehingga tampilan aplikasi menjadi lebih informatif dan responsif.
5	Praktikum ini memberikan pemahaman tentang pengembangan aplikasi web modern yang relevan dengan kebutuhan sistem informasi industri seperti monitoring inventori, laporan kualitas, dan pengelolaan data produksi berbasis web.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React. (2025). <i>React documentation</i> . https://react.dev/
[3]	MDN Web Docs. (2025). <i>Fetch API</i> . https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
[4]	React Router. (2025). <i>React Router Documentation</i> . https://reactrouter.com/

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Wianda Sukandari NIM: 23051430002	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____